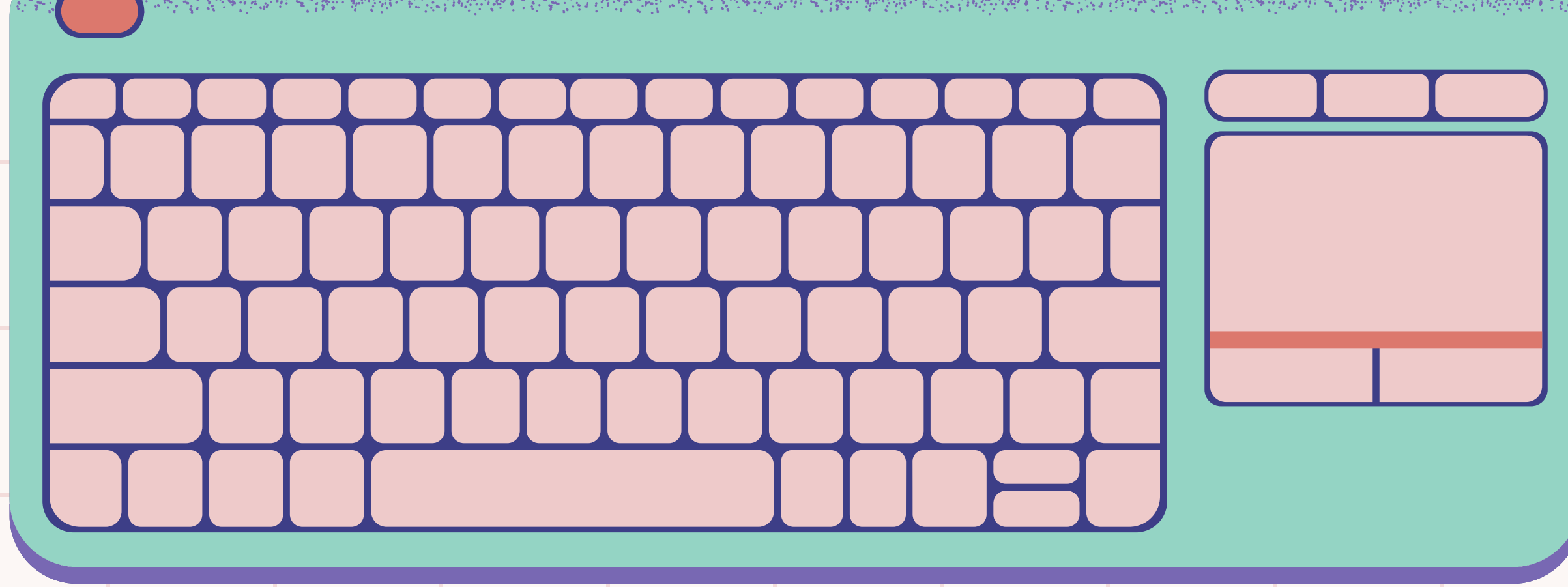
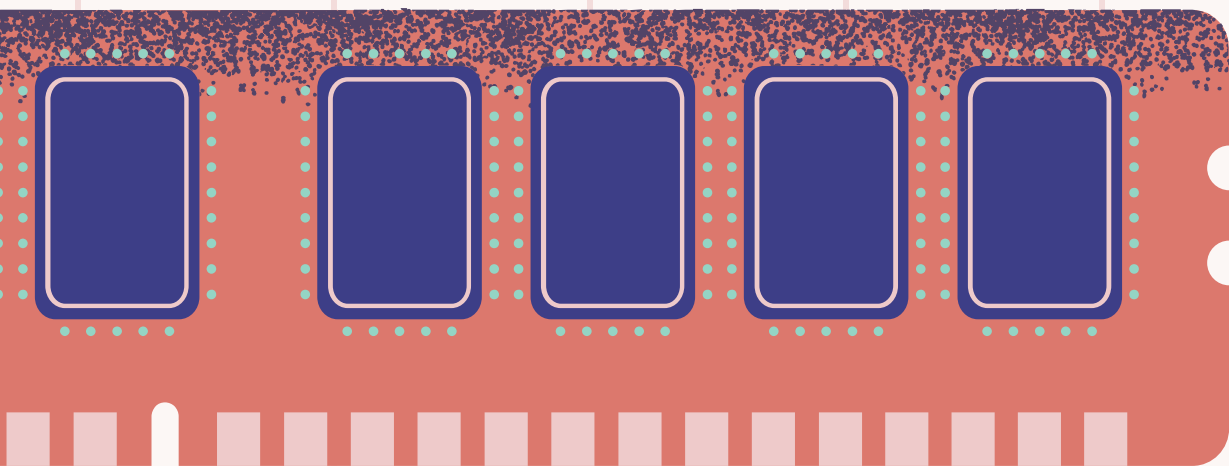




INTRODUCTION TO ABSTRACT FACTORY - GROUP 2 -



0. CONTENTS

01

INTRODUCTORY PROBLEM

02

ABSTRACT FACTORY PATTERN

03

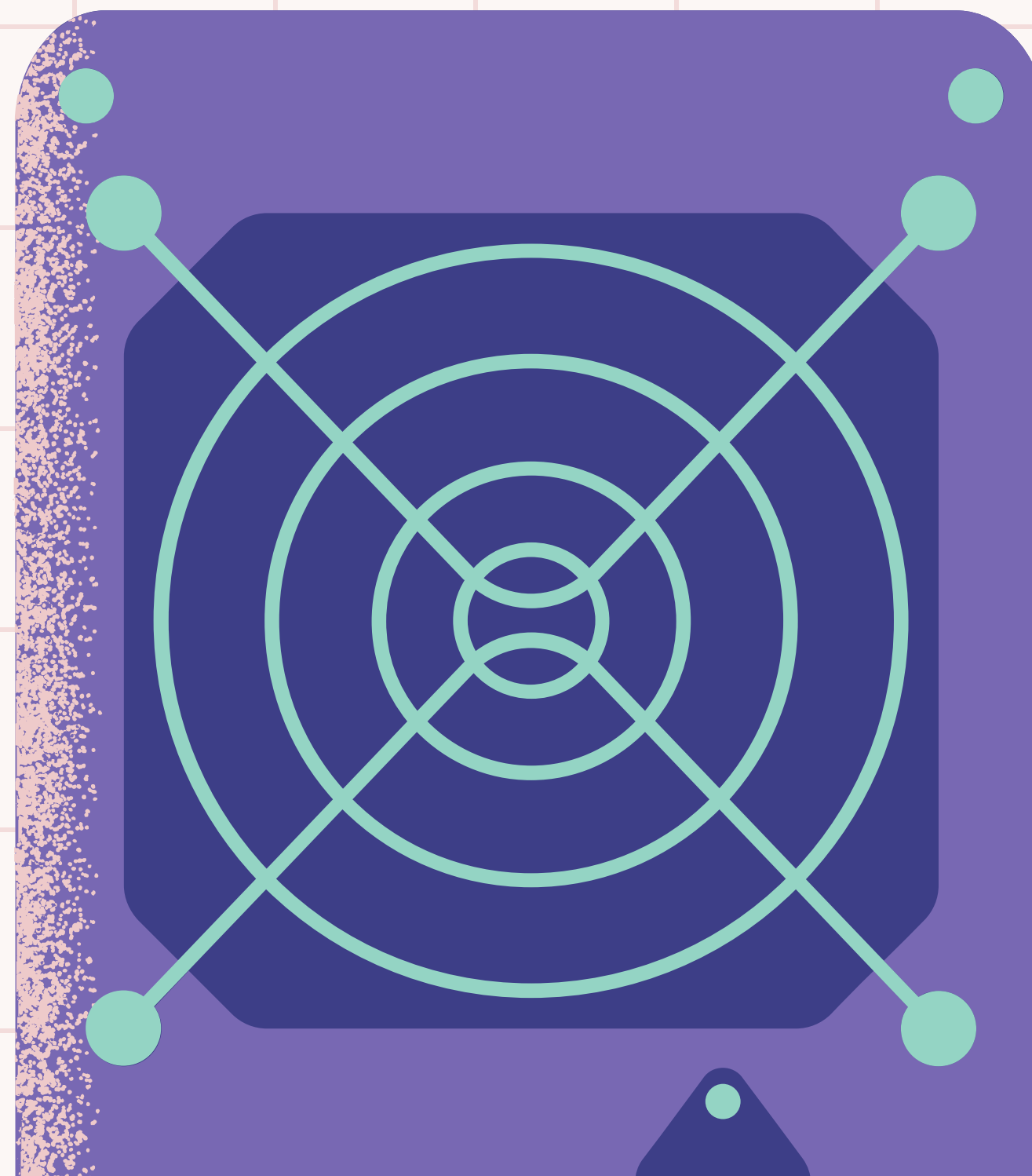
APPLICATIONS

04

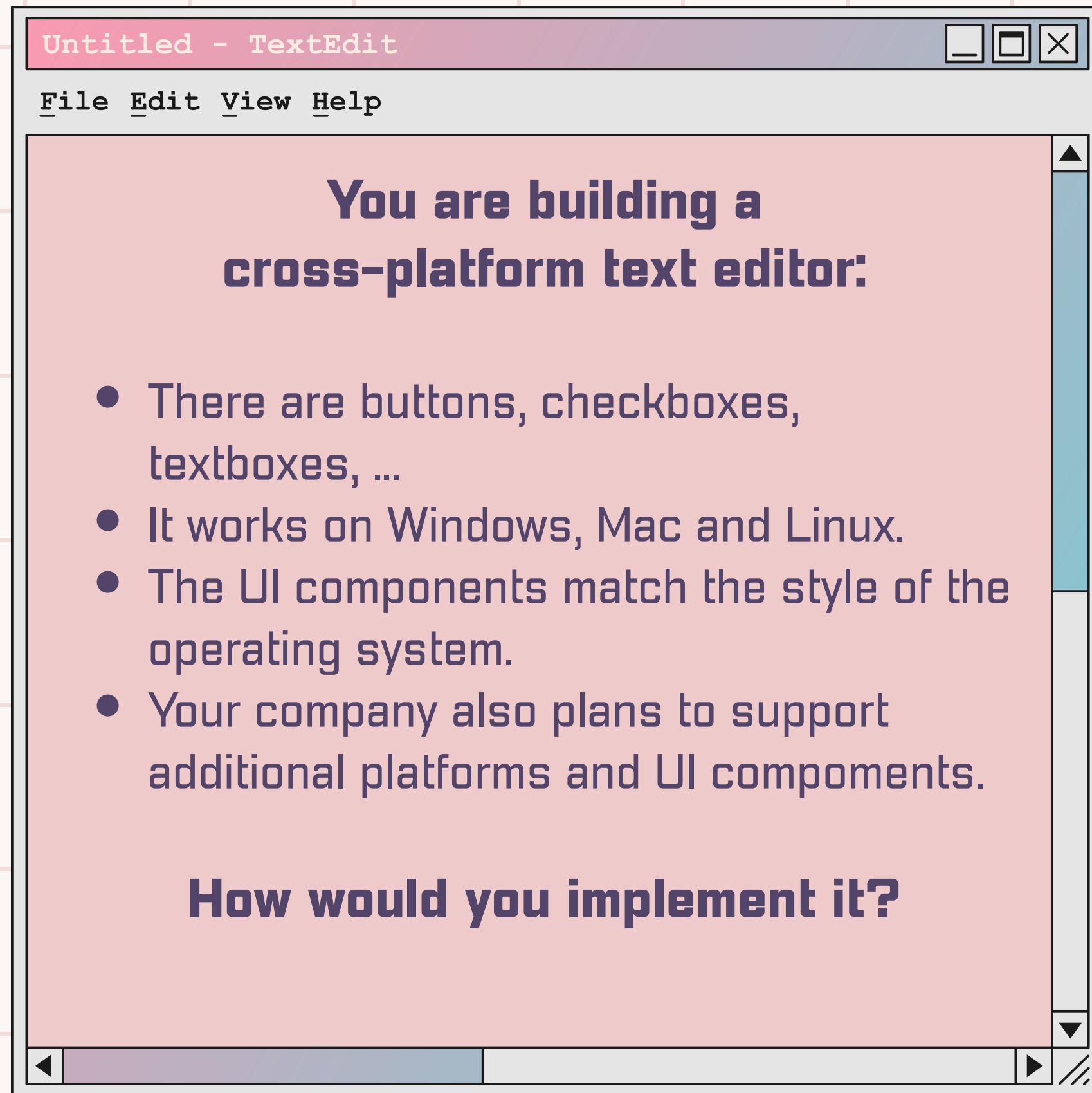
IMPLEMENTATION

05

QUIZZES



1. INTRODUCTION



NAIVE SOLUTION



```
Naive Solution

if (os == "Windows") {
    button = new WindowsButton;
    checkbox = new WindowsCheckbox;
}
else if (os == "Mac") {
    button = new MacButton;
    checkbox = new MacCheckbox;
}
...
```



YOUR FIRST TRY

Hard coding with if-elses whenever an UI component is needed.

PROBLEMS OF THE NAIVE SOLUTION

1 VIOLATION OF OPEN-CLOSED PRINCIPLE

Adding a new platform requires modifying existing code in every place components are created.

2 TIGHT COUPLING

Client code depends directly on concrete classes, making it hard to swap implementations.

3 FAMILY CONSISTENCY NOT GUARANTEED

Nothing prevents mixing components from different families (e.g. WinButton + MacCheckbox).

4 DUPLICATE CREATION LOGIC

The same if-else blocks are scattered across the codebase wherever components are instantiated.

SECOND NAIVE ATTEMPT

Second Naive Attempt

```
// lets abstract the style of the UI component created
Button button = ButtonFactory::create(os);
Checkbox checkbox = CheckboxFactory::create(os);
...
```



SECOND NAIVE ATTEMPT

Creation is moved into factory calls, but the client still chooses the OS and duplicates the pattern for each product type.
=> Cannot fix all problems.

ABSTRACT FACTORY

ABSTRACT PRODUCT

An interface or abstract class for a distinct type of product within a family.
Ex: **Button** or **Checkbox**

CONCRETE PRODUCT

Specific products.
Ex: **WinButton** and **MacButton** implement **Button**.
Ex: **WinCheckbox** and **MacCheckbox** implement **Checkbox**.

ABSTRACT FACTORY

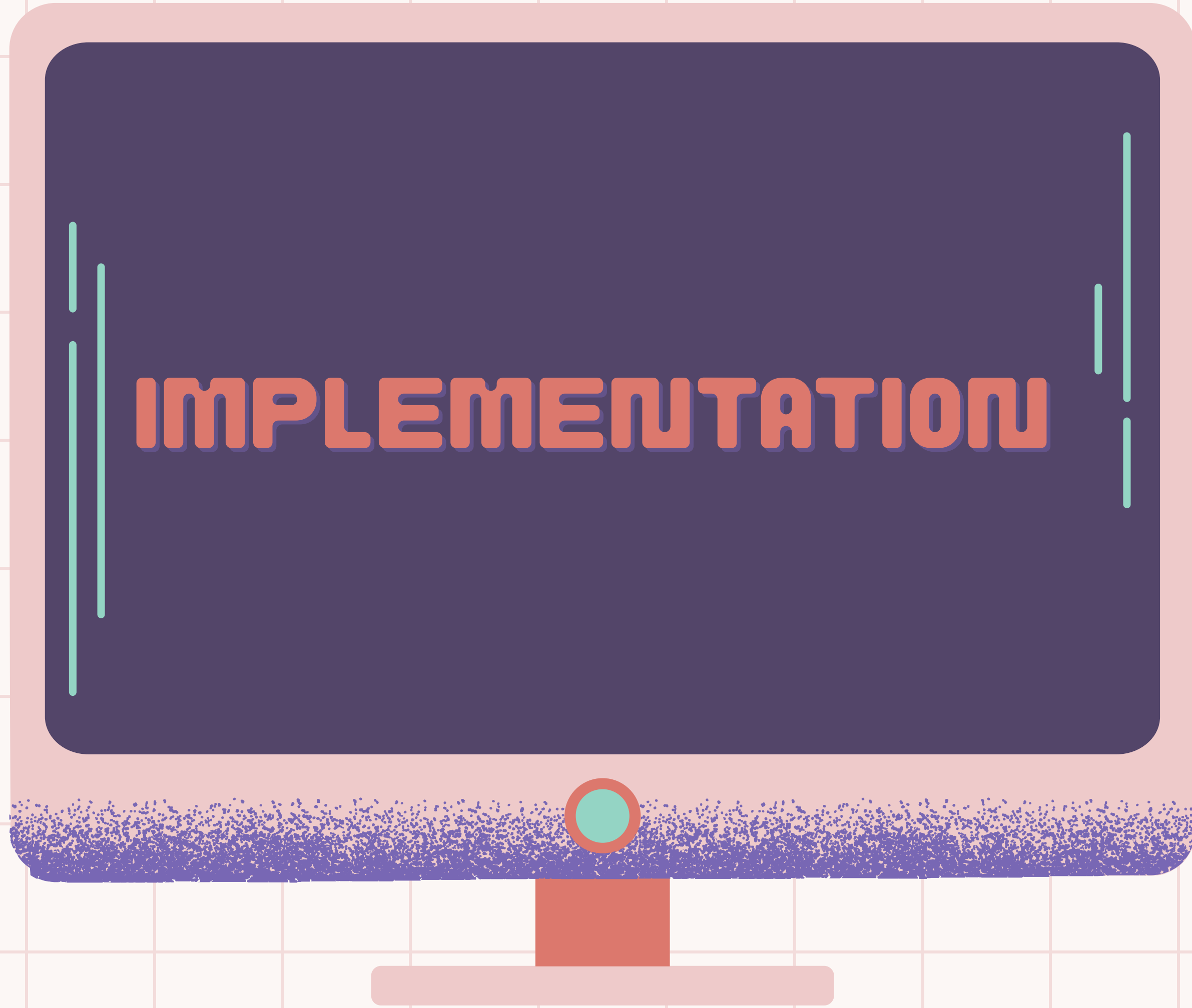
An interface declaring a set of creation methods for each abstract product type
Ex: **createButton()** and **createCheckbox()**

CONCRETE FACTORY

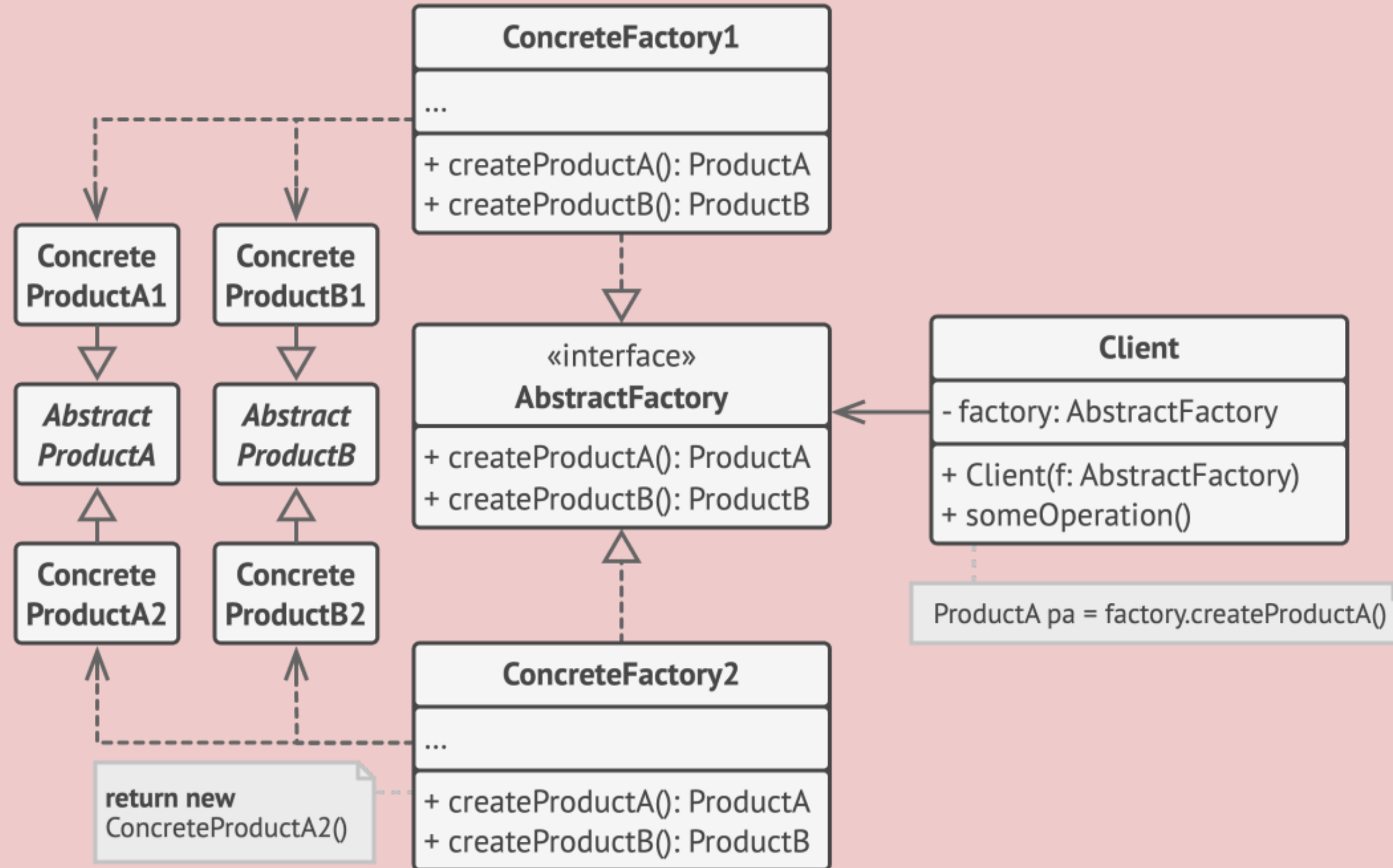
Specific classes that implement the Abstract Factory to produce a concrete variant.
Ex: **WinFactory** creates **WinButton** and **WinCheckbox**

CLIENT

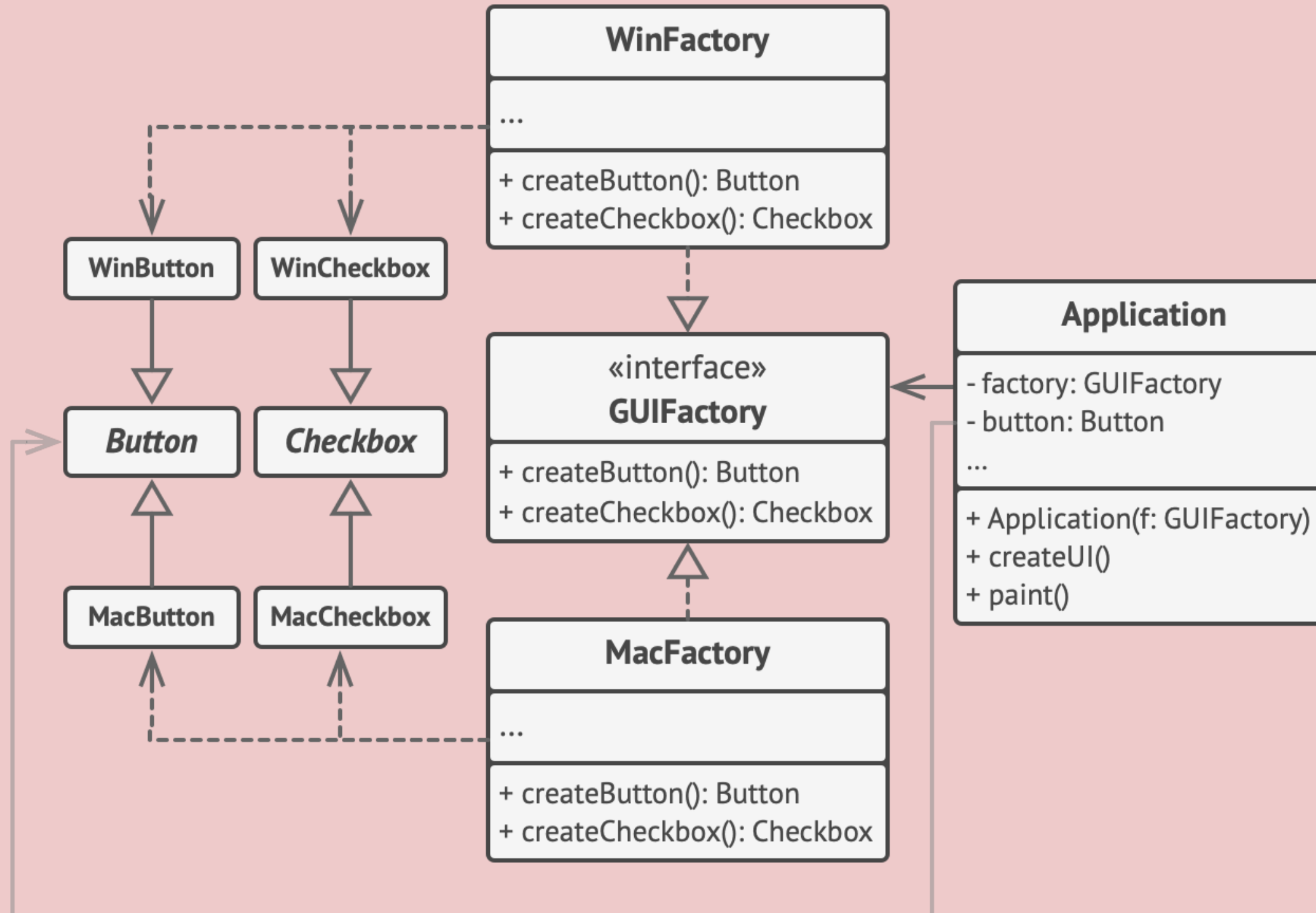
The application code that uses only the interfaces declared by **AbstractFactory** and **AbstractProduct**



General Class Diagram



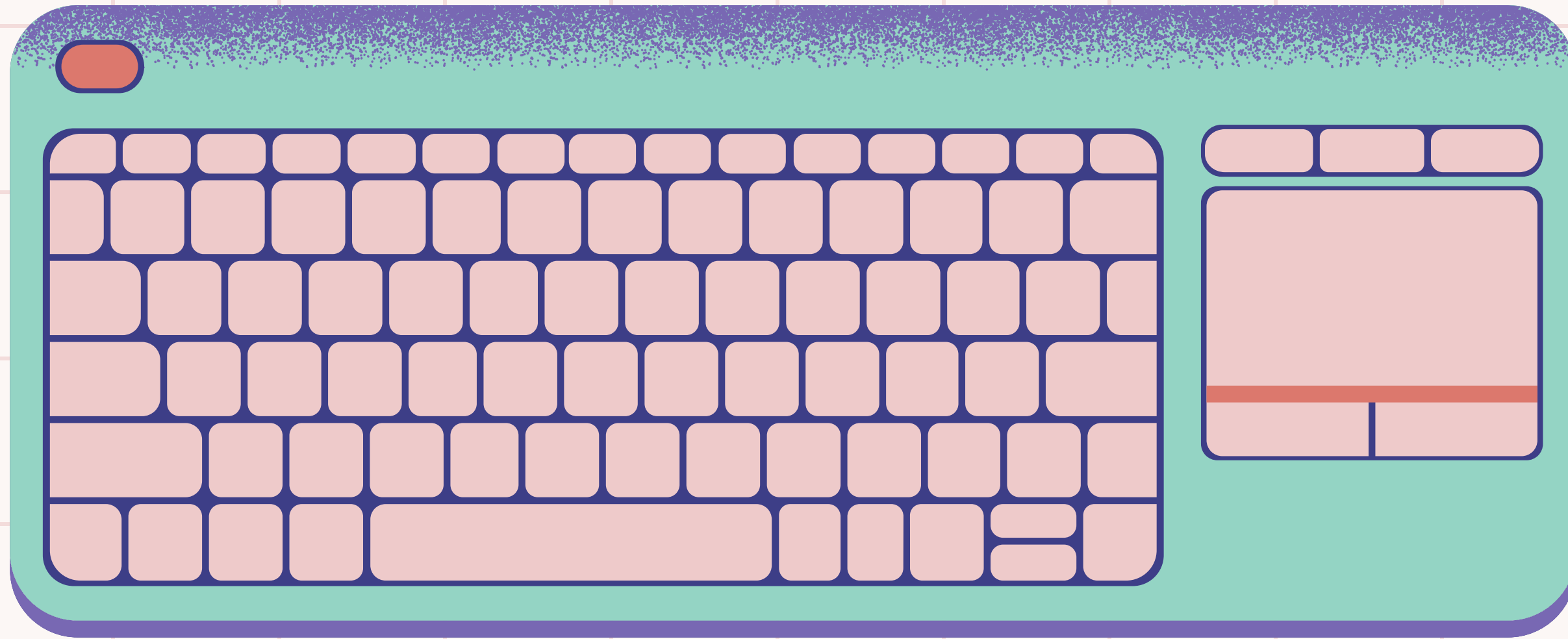
Solution Class Diagram



SAMPLE CODE

SCAN ME





PROS AND CONS

ADVANTAGES

1 DEPENDENCY INVERSION PRINCIPLE

The pattern ensures that client code depends on high-level abstractions rather than concrete implementations.

2 DECOUPLES THE CLIENT

By returning abstract product types rather than concrete classes, the pattern isolates the client from implementation details

3 PRODUCT CONSISTENCY

The pattern is designed to create families of related objects, ensuring to receive compatible objects from the same family.

4 EASE OF ADDING PRODUCT FAMILIES

Adding a new product families, such as a new OS GUI factory, is easy and doesn't require changes to existing client code.

DISADVANTAGES

1 NEW PRODUCT TYPES

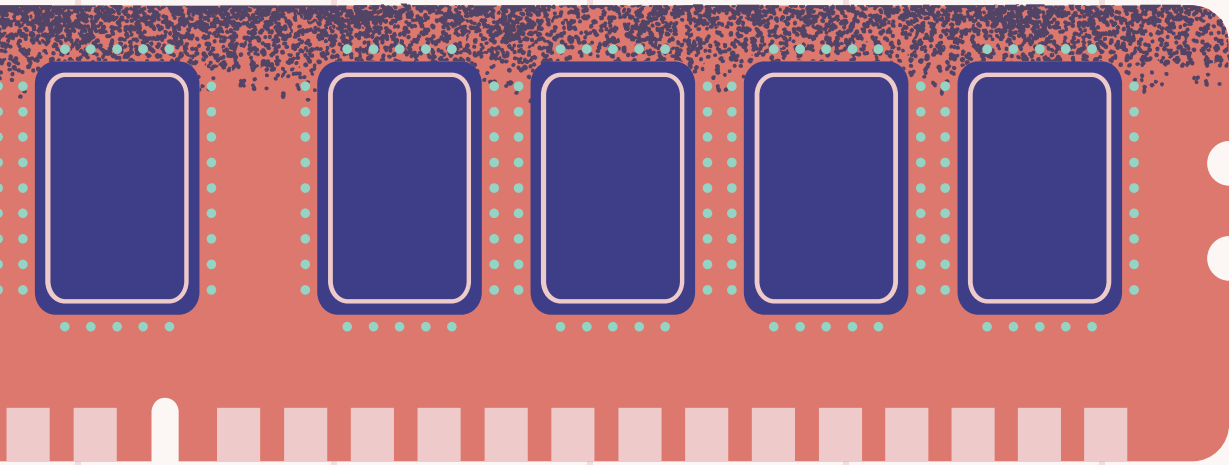
Requiring altering the main Abstract Factory interface and cascading those changes across all existing concrete factory

2 INCREASED SYSTEM COMPLEXITY

The pattern introduces many new interfaces and classes into the codebase, make the system more complex

3 UPFRONT DESIGN OVERHEAD

Requiring correctly predicting the variations of product families early in the design phase. Or the structure can become a nightmare



04

OTHER APPLICATIONS

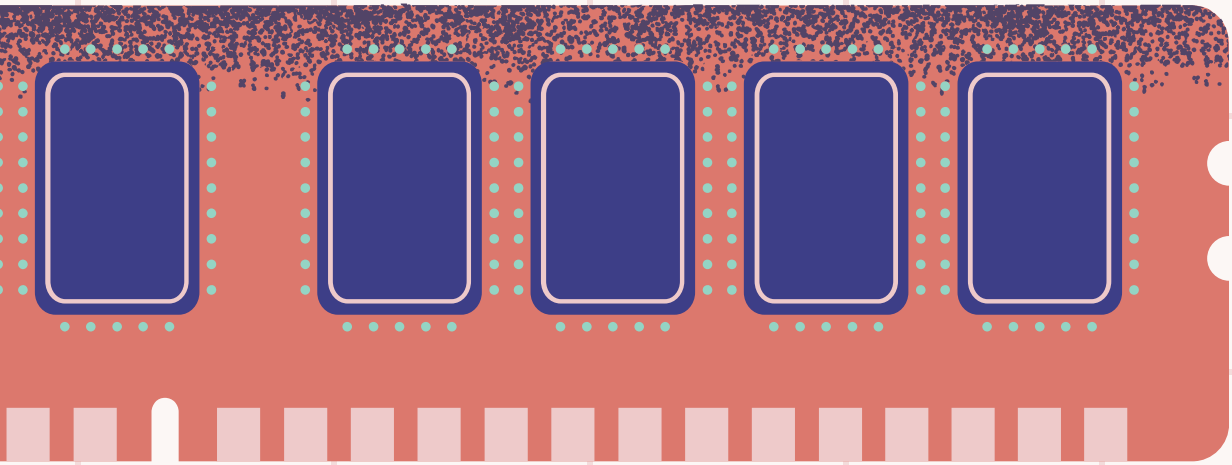
Learn More



Game Development & Level Theming



- Abstract Factory:
 - **BiomeFactory**
- Products:
 - **Enemy**
 - **Weapon**
- Concrete Factories:
 - **IceFactory**
 - **NetherFactory**



05

QUIZESS

Learn More



1. What is the main purpose of the Abstract Factory Pattern?

A. Ensure a class has only one instance.

B. Create families of related objects without specifying their concrete classes.

C. Separate complex object construction.

D. Change an object's behavior.

1. What is the main purpose of the Abstract Factory Pattern?

A. Ensure a class has only one instance.

B. Create families of related objects without specifying their concrete classes.

C. Separate complex object construction.

D. Change an object's behavior.

2. How does Abstract Factory differ from Factory Method?

A. Factory Method creates families; Abstract Factory creates single objects.

B. Abstract Factory uses inheritance; Factory Method uses composition.

C. Abstract Factory creates product families; Factory Method creates a single product type.

D. They are identical.

2. How does Abstract Factory differ from Factory Method?

A. Factory Method creates families; Abstract Factory creates single objects.

B. Abstract Factory uses inheritance; Factory Method uses composition.

C. Abstract Factory creates product families; Factory Method creates a single product type.

D. They are identical.

3. What is a major disadvantage of Abstract Factory?

A. It tightly couples client code to concrete classes.

B. Adding new product types requires changing the factory interface and all subclasses.

C. Adding new product families is nearly impossible.

D. It exposes instantiation logic to the client.

3. What is a major disadvantage of Abstract Factory?

A. It tightly couples client code to concrete classes.

B. Adding new product types requires changing the factory interface and all subclasses.

C. Adding new product families is nearly impossible.

D. It exposes instantiation logic to the client.

4. Which scenario is best suited for Abstract Factory?

A. A cross-platform UI toolkit with OS-specific buttons, checkboxes, and text fields.

B. A logger writing to either a file or a database.

C. A document parser building a DOM tree step-by-step.

D. A caching system cloning objects.

4. Which scenario is best suited for Abstract Factory?

A. A cross-platform UI toolkit with OS-specific buttons, checkboxes, and text fields.

B. A logger writing to either a file or a database.

C. A document parser building a DOM tree step-by-step.

D. A caching system cloning objects.

5. Which component actually creates concrete objects?

A. The Client

B. The Abstract Factory Interface

C. The Concrete Factories

D. The Abstract Products

5. Which component actually creates concrete objects?

A. The Client

B. The Abstract Factory Interface

C. The Concrete Factories

D. The Abstract Products

6. Which pattern is often combined with Concrete Factories?

A. Observer

B. Singleton

C. Strategy

D. Decorator

6. Which pattern is often combined with Concrete Factories?

A. Observer

B. Singleton

C. Strategy

D. Decorator

7. What should factory creation methods return?

A. Concrete types (e.g., WindowsButton)

B. Abstract product interfaces (e.g., IButton)

C. Generic objects (Object, Any)

D. Boolean success flags

7. What should factory creation methods return?

A. Concrete types (e.g., WindowsButton)

B. Abstract product interfaces (e.g., IButton)

C. Generic objects (Object, Any)

D. Boolean success flags

8. Which SOLID principle does Abstract Factory promote most?

A. Single Responsibility Principle

B. Open/Closed Principle

C. Liskov Substitution Principle

D. Dependency Inversion Principle

8. Which SOLID principle does Abstract Factory promote most?

A. Single Responsibility Principle

B. Open/Closed Principle

C. Liskov Substitution Principle

D. Dependency Inversion Principle

ABSTRACT FACTORY VS. FACTORY METHOD

1 ABSTRACT FACTORY

- **Scope:** Creates families of related objects without specifying their concrete classes
- **Mechanism:** Relies on object composition; creation is delegated to multiple factory methods exposed in an interface.
- **Design Level:** Operates at a higher level of abstraction. It is often implemented using a set of Factory Methods.

2 FACTORY METHOD

- **Scope:** Creates single specific objects or products.
- **Mechanism:** Relies on inheritance; it defines an interface for creating an object, but lets subclasses decide which class to instantiate.
- **Design Level:** Operates strictly at the method level.

ABSTRACT FACTORY VS ADAPTER

ABSTRACT FACTORY

Abstract Factory is a **Creational pattern**. It creates families of related objects without specifying concrete classes.

ADAPTER

Adapter is a **Structural pattern** that wraps an incompatible class with the interface a client expects.



THANK YOU

For Your Attention

